

ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



MÔN:
PHÁT TRIỂN ỨNG DỤNG IoT

ĐỀ TÀI:
XÂY DỰNG ĐỒNG HỒ THỜI GIAN THỰC VỚI
ESP32 VÀ NTP

Giáo viên hướng dẫn: Võ Viết Dũng

Sinh viên thực hiện : TRẦN KHÁNH

Mã sinh viên: 22T1020640

Lớp: CNTT - K46H

Chuyên ngành: Mạng máy tính

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

- 1.1. Đặt vấn đề và tính cấp thiết của đề tài
- 1.2. So sánh giải pháp RTC vật lý và giao thức NTP

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

- 2.1. Vi điều khiển ESP32 và khả năng kết nối Wi-Fi
- 2.2. Giao thức NTP (Network Time Protocol)
- 2.3. Công nghệ hiển thị OLED và chuẩn giao tiếp I2C

CHƯƠNG 3: THIẾT KẾ VÀ SƠ ĐỒ NGUYÊN LÝ

- 3.1. Sơ đồ khối và nguyên lý kết nối phần cứng
- 3.2. Sơ đồ đấu nối chi tiết
- 3.3. Xây dựng môi trường mô phỏng trên Wokwi

CHƯƠNG 4: THUẬT TOÁN VÀ LẬP TRÌNH

- 4.1. Triển khai mã nguồn và giải thích các hàm chính
- 4.2. Kết quả hiển thị và độ chính xác của hệ thống
- 4.3 Kết luận

TÀI LIỆU THAM KHẢO

LỜI NÓI ĐẦU

Trong kỷ nguyên của Cách mạng Công nghiệp 4.0, Internet vạn vật (IoT) đã và đang trở thành nền tảng cốt lõi trong việc kết nối và tự động hóa các hệ thống điện tử. Trong số đó, việc quản lý thời gian chính xác là một yêu cầu tiên quyết đối với các hệ thống giám sát, điều khiển và thu thập dữ liệu. Tuy nhiên, các giải pháp sử dụng mô-đun RTC (Real-Time Clock) truyền thống thường gặp phải những hạn chế như sai số tích lũy theo thời gian hoặc yêu cầu duy trì nguồn pin dự phòng phức tạp.

Để giải quyết vấn đề này, xu hướng hiện nay là tận dụng hạ tầng mạng Internet để đồng bộ hóa thời gian thông qua giao thức NTP (Network Time Protocol). Với sự hỗ trợ mạnh mẽ từ vi điều khiển **ESP32** – một dòng chip tích hợp sẵn Wi-Fi và Bluetooth với hiệu năng cao – việc xây dựng một hệ thống thời gian thực có độ chính xác tuyệt đối trở nên khả thi và tối ưu hơn về mặt chi phí.

Đề tài "Xây dựng đồng hồ thời gian thực với ESP32 và NTP"

được thực hiện nhằm nghiên cứu phương thức kết nối và lấy dữ liệu thời gian từ các máy chủ trên toàn cầu. Qua đó, người thực hiện sẽ đi sâu vào việc làm chủ các công nghệ:

- Cấu hình kết nối không dây trên nền tảng ESP32.
- Tìm hiểu cơ chế hoạt động của giao thức NTP để xử lý độ trễ mạng và múi giờ.
- Kỹ thuật hiển thị dữ liệu trực quan lên màn hình OLED và tối ưu hóa mã nguồn trên các môi trường giả lập cũng như thực tế.

Mặc dù quy mô đề tài tập trung vào việc hiển thị thời gian, nhưng đây là bước đệm quan trọng để phát triển các ứng dụng IoT phức tạp hơn

nhu hệ thống tưới tiêu tự động, thiết bị điểm danh thông minh hay các bộ điều khiển công nghiệp cần tính đồng bộ cao.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề và tính cấp thiết của đề tài

Trong kỷ nguyên Internet vạn vật (IoT), thời gian thực (Real-Time) đóng vai trò là "xương sống" cho các kịch bản vận hành tự động. Một hệ thống Smart Home không thể hoạt động hiệu quả nếu việc bật đèn sân vườn lúc 18:00 hay kích hoạt hệ thống tưới cây lúc 05:00 bị sai lệch. Sự chính xác về thời gian đảm bảo tính đồng bộ giữa các thiết bị trong mạng, giúp việc ghi nhật ký dữ liệu (logging) và thực thi các tác vụ theo lịch trình trở nên tin cậy.

1.2. So sánh giải pháp RTC vật lý và giao thức NTP

Hiện nay, có hai phương pháp phổ biến để tích hợp thời gian thực vào vi điều khiển:

1.2.1. Sử dụng Module RTC vật lý (như DS3231, DS1307)

- **Ưu điểm:** Duy trì thời gian ngay cả khi không có kết nối Internet nhờ pin dự phòng.
- **Nhược điểm:** Tốn thêm chi phí mua linh kiện; mạch điện trở nên cồng kềnh hơn; pin dự phòng sau một thời gian sẽ yếu dẫn đến sai lệch giờ; người dùng phải thiết lập giờ thủ công lần đầu.

1.2.2. Sử dụng giao thức NTP (Network Time Protocol)

- **Ưu điểm:** Không tốn chi phí linh kiện bổ sung; tự động đồng bộ theo giờ chuẩn quốc tế qua Wi-Fi; tự động cập nhật giờ mùa hè (DST) và múi giờ.
- **Lý do chọn NTP cho đề tài:** Với sự phổ biến của Wi-Fi, việc sử dụng NTP giúp hệ thống nhỏ gọn, hiện đại và luôn chính xác tuyệt đối mà không cần can thiệp thủ công.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Vi điều khiển ESP32 và khả năng kết nối Wi-Fi

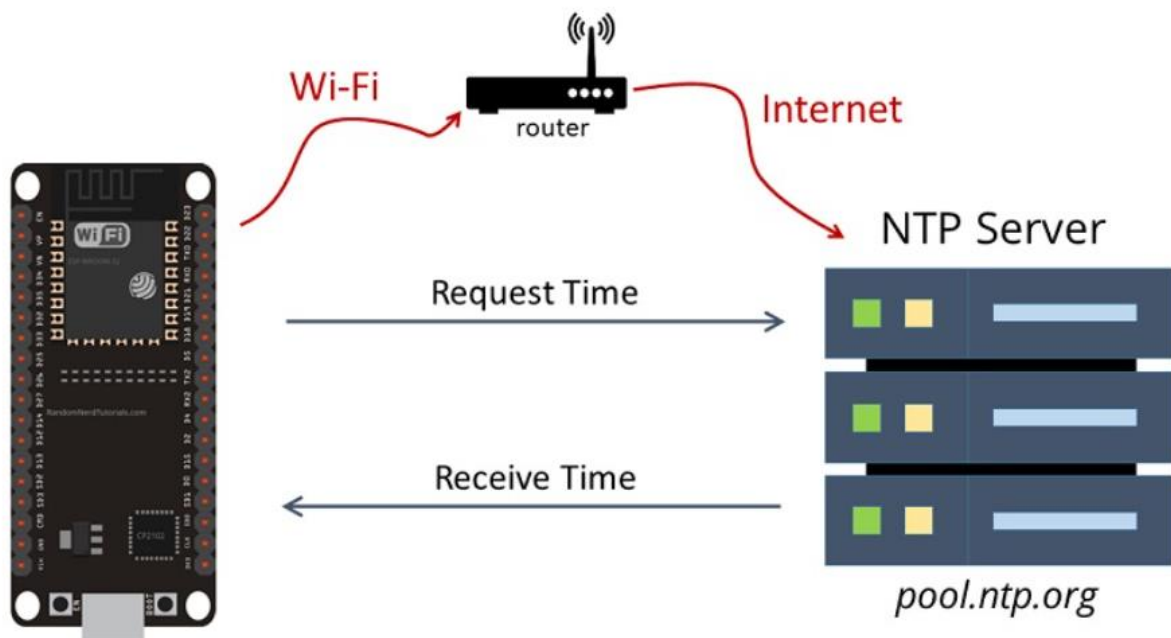
ESP32 là dòng vi điều khiển công suất thấp, tích hợp sẵn Wi-Fi và Bluetooth kép (Classic và BLE), được sản xuất bởi Espressif Systems.

- **Kiến trúc:** Sử dụng lõi Tensilica Xtensa® Dual-Core 32-bit LX6, tốc độ lên đến 240 MHz.
- **Bộ nhớ:** 520 KB SRAM, hỗ trợ bộ nhớ Flash ngoài lên tới 16 MB.
- **Ưu điểm trong đề tài:** Khả năng kết nối Wi-Fi mạnh mẽ và thư viện hỗ trợ NTP phong phú giúp việc lập trình trở nên đơn giản.

2.2. Giao thức NTP (Network Time Protocol)

NTP (Network Time Protocol) là giao thức mạng giúp đồng bộ hóa thời gian giữa các hệ thống máy tính trong mạng.

Một trong các NTP Server phổ biến là `pool.ntp.org`, bất kỳ ai cũng có thể yêu cầu thời gian từ Server này với tư cách là Client.



- **Cấu trúc Client-Server:** ESP32 đóng vai trò là **NTP Client**. Nó gửi một gói tin yêu cầu qua giao thức **UDP** (cổng 123) tới **NTP Server** (ví dụ: `pool.ntp.org`).

- **Unix Timestamp:** Server phản hồi một gói tin chứa số giây đã trôi qua kể từ 00:00:00 UTC ngày 1 tháng 1 năm 1970 (gọi là Unix Epoch). Từ số giây này, vi điều khiển sẽ tính toán ra Ngày/Tháng/Năm/Giờ/Phút/Giây hiện tại.

- **Máy chủ NTP**

Pool.ntp.org nơi chứa dữ liệu thời gian toàn cầu trên máy chủ và bất kỳ ai cũng có thể yêu cầu thời gian địa phương của họ bằng cách sử dụng hồ bơi này. Dưới đây là một số máy chủ khác mà chúng tôi có thể truy cập:

Khu vực	Tên máy chủ
Toàn cầu	pool.ntp.org
Châu Á	asia.pool.ntp.org
Châu Âu	europa.pool.ntp.org
Bắc Mỹ	north-america.pool.ntp.org
Châu Đại Dương	oceania.pool.ntp.org
Nam Mỹ	Nam Mỹ.pool.ntp.org

2.3. Công nghệ hiển thị OLED và chuẩn giao tiếp I2C

Màn hình OLED (Organic Light Emitting Diode) 0.96 inch là lựa chọn tối ưu cho các dự án nhỏ gọn.

- **Công nghệ:** Tự phát sáng, không cần đèn nền, độ tương phản cao và tiết kiệm điện.
- **Giao tiếp I2C:** Chỉ sử dụng 2 dây tín hiệu là **SDA** (Serial Data) và **SCL** (Serial Clock). Điều này cực kỳ quan trọng đối với ESP32 vì nó giúp tiết kiệm tối đa các chân GPIO cho các cảm biến hoặc thiết bị ngoại vi khác.

CHƯƠNG 3: THIẾT KẾ VÀ SƠ ĐỒ NGUYÊN LÝ

3.1. Sơ đồ khối và nguyên lý kết nối phần cứng

Trong phần này, bạn nên vẽ một sơ đồ gồm 4 khối chính. Việc giải thích từng khối sẽ giúp bạn chiếm khoảng 1 trang giấy.

- **Khối nguồn:** Sử dụng nguồn 5V từ cổng USB của máy tính hoặc Adapter. ESP32 sẽ hạ áp xuống 3.3V để cung cấp cho màn hình OLED.
- **Khối xử lý trung tâm (ESP32):** Trái tim của hệ thống, chịu trách nhiệm kết nối Wi-Fi, xử lý giao thức UDP và điều khiển hiển thị.
- **Khối truyền thông (Wi-Fi & NTP):** Giải thích cách ESP32 đóng vai trò là một NTP Client gửi yêu cầu đến các Server như time.google.com hoặc pool.ntp.org.
- **Khối hiển thị (OLED 0.96 inch):** Sử dụng chuẩn giao tiếp I2C để nhận dữ liệu từ ESP32 và hiển thị lên màn hình.

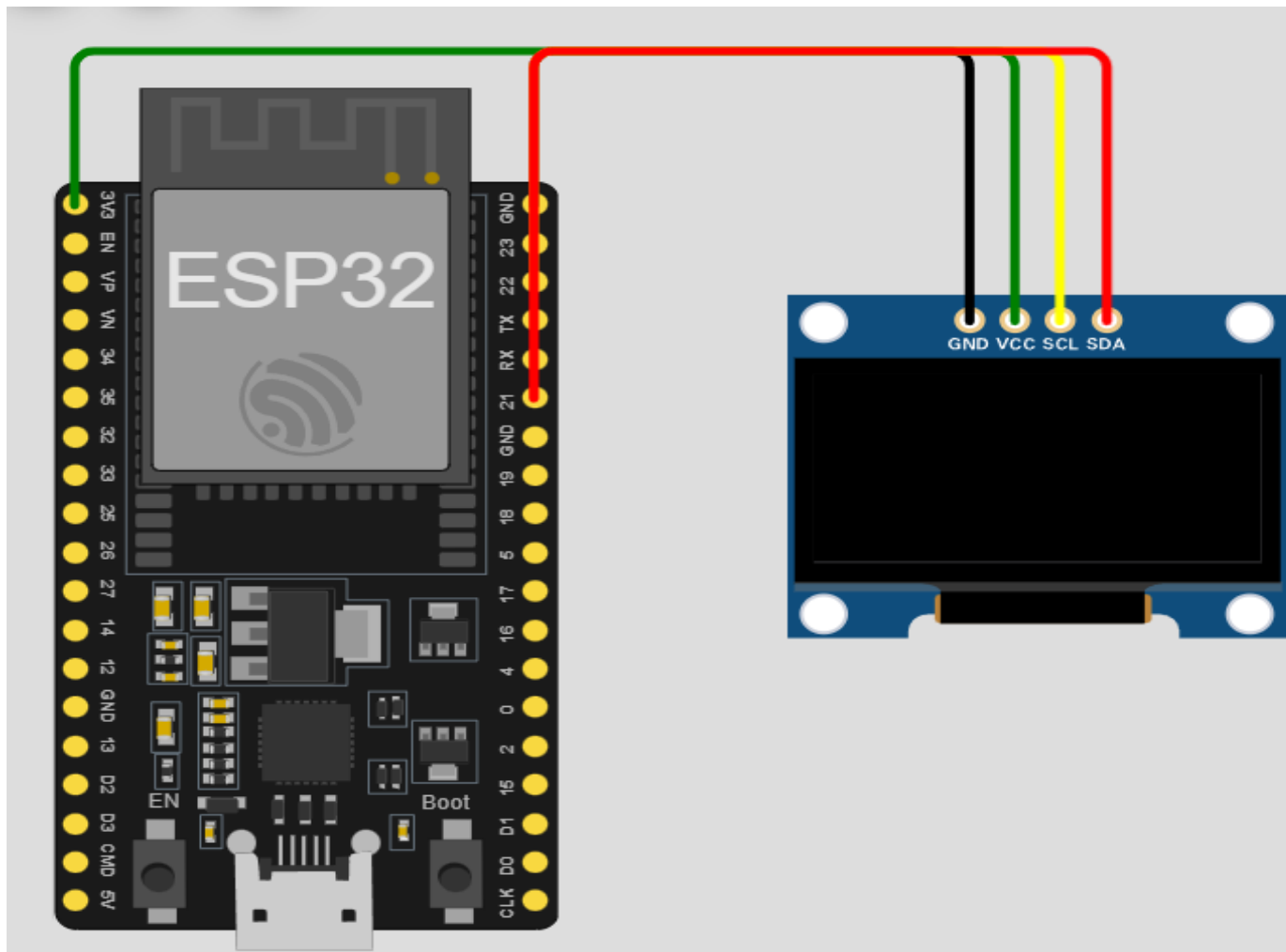
3.2. Sơ đồ đấu nối chi tiết

Việc kết nối giữa ESP32 và màn hình OLED SSD1306 được thực hiện qua giao thức I2C theo sơ đồ sau:

Chân OLED	Chân ESP32	Chức năng
VCC	3V3	Cấp nguồn dương 3.3V
GND	GND	Nối đất (Cực âm)
SCL	GPIO 22	Chân xung giữ nhịp dữ liệu I2C
SDA	GPIO 21	Chân truyền nhận dữ liệu I2C

3.3. Xây dựng môi trường mô phỏng trên Wokwi

Sơ đồ wokwi:



Sơ đồ thể hiện việc kết nối tín hiệu và nguồn giữa vi điều khiển ESP32 và màn hình OLED 0.96 inch. Việc sử dụng giao thức I2C giúp tối giản số lượng dây dẫn, chỉ cần 4 kết nối chính:

1. Kết nối nguồn (Power):

Dây đỏ (VCC): Chân **VCC** của màn hình OLED được nối với chân **3V3** trên board ESP32 để cấp nguồn dương 3.3V.

Dây đen (GND): Chân **GND** của màn hình OLED được nối trực tiếp với chân **GND** của ESP32 để tạo điểm tiếp đất chung cho hệ thống

2. Kết nối tín hiệu (Data & Clock):

Dây vàng (SCL - Serial Clock): Chân **SCL** của màn hình OLED được kết nối với chân **GPIO 22** của ESP32. Đây là chân giữ nhịp xung đồng hồ cho quá trình truyền dữ liệu I2C.

Dây xanh lá (SDA - Serial Data): Chân **SDA** của màn hình OLED được kết nối với chân **GPIO 21** của ESP32. Đây là đường truyền dữ liệu hai chiều giữa vi điều khiển và màn hình.

CHƯƠNG 4: THUẬT TOÁN VÀ LẬP TRÌNH

4.1. Triển khai mã nguồn và giải thích các hàm chính

Chương trình code lấy thời gian cho ESP32 NTP

Dưới đây là đoạn code mẫu để ESP32 NTP lấy thông tin ngày và giờ từ NTP Server và in kết quả ra màn hình :

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include <time.h>
// — Cấu hình Wi-Fi



---


#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASS ""
// — OLED
#define SCREEN_W 128
#define SCREEN_H 64
#define OLED_ADDR 0x3C
Adafruit_SSD1306 display(SCREEN_W, SCREEN_H, &Wire, -1);
// — NTP
#define NTP_SERVER "pool.ntp.org"
#define GMT_OFFSET 25200 // UTC+7 (Ho Chi Minh City)
#define DST_OFFSET 0
// Thêm/xoá giờ phút tùy ý (HH, MM)
struct Schedule { int hour; int minute; const char* msg; };
Schedule schedules[] = {
```

```

    { 7, 0, "Chao buoi sang!" },
    { 12, 0, "Den gio an trua" },
    { 18, 0, "Tan ca roi!" },
    { 22, 0, "Di ngu thoi :)" },
};

const int NUM_SCHED = sizeof(schedules) / sizeof(schedules[0]);
int lastNotifMinute = -1; // tránh thông báo trùng
// — Tên thứ & tháng

```

```

const char* DOW[] = {"CN","T2","T3","T4","T5","T6","T7"};
const char* MON[] = {"","Jan","Feb","Mar","Apr","May","Jun",
                    "Jul","Aug","Sep","Oct","Nov","Dec"};

void connectWiFi() {
    Serial.printf("Ket noi Wi-Fi: %s\n", WIFI_SSID);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 20);
    display.print("Connecting WiFi...");
    display.display();
    int tries = 0;
    while (WiFi.status() != WL_CONNECTED && tries++ < 40) {
        delay(500); Serial.print(".");
    }
    Serial.println(WiFi.status() == WL_CONNECTED ? "\nWiFi OK!"
: "\nWiFi FAIL");
}

void syncNTP() {
    Serial.println("Dong bo NTP...");
    configTime(GMT_OFFSET, DST_OFFSET, NTP_SERVER);
    struct tm t;
    int tries = 0;

```

```

while (!getLocalTime(&t) && tries++ < 20) delay(500);
if (tries < 20)
    Serial.printf("Thoi gian: %02d:%02d:%02d %02d/%02d/%04d\n",
        t.tm_hour, t.tm_min, t.tm_sec,
        t.tm_mday, t.tm_mon+1, t.tm_year+1900);
else
    Serial.println("NTP that bai!");
}

void drawClock(struct tm &t) {
    display.clearDisplay();
    // — Đồng hồ lớn (HH:MM:SS)

    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    char timeBuf[9];
    sprintf(timeBuf, "%02d:%02d:%02d", t.tm_hour, t.tm_min,
t.tm_sec);
    int x = (SCREEN_W - strlen(timeBuf) * 12) / 2;
    display.setCursor(x, 2);
    display.print(timeBuf);
    // — Đường kẻ ngang
    display.drawFastHLine(0, 22, SCREEN_W, SSD1306_WHITE);
    // — Ngày / Thứ
    display.setTextSize(1);
    char dateBuf[20];
    sprintf(dateBuf, "%s %02d/%02d/%04d",
        DOW[t.tm_wday],
        t.tm_mday, t.tm_mon+1, t.tm_year+1900);
    x = (SCREEN_W - strlen(dateBuf) * 6) / 2;
    display.setCursor(x, 26);
    display.print(dateBuf);
    // — Thanh tiến trình giây

```

```
display.drawRect(0, 37, SCREEN_W, 6, SSD1306_WHITE);
int barW = map(t.tm_sec, 0, 59, 0, SCREEN_W - 2);
display.fillRect(1, 38, barW, 4, SSD1306_WHITE);
// — Trạng thái kết nối
display.setCursor(0, 48);
display.print(WiFi.status() == WL_CONNECTED ? "WiFi: OK" :
"WiFi: --");
// — Sync interval (mỗi giờ)
```

```
display.setCursor(70, 48);
display.print("NTP:OK");
display.setCursor(0, 57);
display.printf("UTC+7 | %s", NTP_SERVER);
display.display();
}
void checkSchedule(struct tm &t) {
    int nowMin = t.tm_hour * 60 + t.tm_min;
    if (nowMin == lastNotifMinute) return; // đã thông báo phút này rồi
    for (int i = 0; i < NUM_SCHED; i++) {
        if (t.tm_hour == schedules[i].hour && t.tm_min ==
schedules[i].minute) {
            lastNotifMinute = nowMin;
            Serial.printf("[THONG BAO %02d:%02d] %s\n",
                schedules[i].hour, schedules[i].minute, schedules[i].msg);
            // Hiển thị thông báo trên OLED 3 giây
            display.clearDisplay();
            display.setTextSize(1);
            display.setCursor(0, 0);
            display.print(">> THONG BAO <<");
            display.drawFastHLine(0, 10, SCREEN_W, SSD1306_WHITE);
            display.setCursor(0, 18);
            display.setTextSize(1);
            display.print(schedules[i].msg);
```

```

    display.setCursor(0, 40);
    display.printf("%02d:%02d", t.tm_hour, t.tm_min);
    display.display();
    delay(3000);
    break;
}
}
}

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22); // SDA=21, SCL=22
    // Khởi OLED
    if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
        Serial.println("OLED không tìm thấy!");
        while (true);
    }
    display.clearDisplay();
    display.display();
    connectWiFi();
    syncNTP();
}

unsigned long lastSync = 0;
const unsigned long SYNC_INTERVAL = 3600000UL; // 1 tiếng
void loop() {
    struct tm t;
    if (!getLocalTime(&t)) { delay(1000); return; }
    drawClock(t);
    checkSchedule(t);
    // Re-sync NTP mỗi giờ
    if (millis() - lastSync > SYNC_INTERVAL) {
        syncNTP();
        lastSync = millis();
    }
}

```

```
delay(1000);  
}
```

***Giải thích chương trình:**

1. Khai báo thư viện để ESP32 kết nối với WiFi và có thể lấy thời gian:

```
#include <Wire.h>  
#include <Adafruit_GFX.h>  
#include <Adafruit_SSD1306.h>  
#include <WiFi.h>  
#include <time.h>
```

Đặt thông tin cho mạng WiFi

```
#define WIFI_SSID "wokwi-GUEST"  
#define WIFI_PASS ""
```

2. Cài đặt thời gian và NTP Server

***NTP Sever**

Như đã nói, ESP32 sẽ yêu cầu thời gian từ máy chủ Server có tên là pool.ntp.org. Đây là một máy chủ về thời gian mà bất kỳ ai cũng có thể yêu cầu thời gian từ nó:

Xác định múi giờ của mình, như ở Việt Nam thì múi giờ là 7:

***GMT_OFFSET (25200):** Chính là khoảng cách 7 tiếng đã quy đổi ra giây.

***DST_OFFSET (0):** Đây là "Daylight Saving Time" (Giờ mùa hè). Ở Việt Nam, chúng ta không áp dụng quy tắc đổi giờ theo mùa, nên giá trị này luôn luôn bằng **0**. Nếu bạn ở các nước như Mỹ hoặc Châu Âu, con số này sẽ thay đổi thành 3600 (1 giờ) vào mùa hè.

```
#define NTP_SERVER "pool.ntp.org"  
#define GMT_OFFSET 25200  
#define DST_OFFSET 0
```

In thời gian

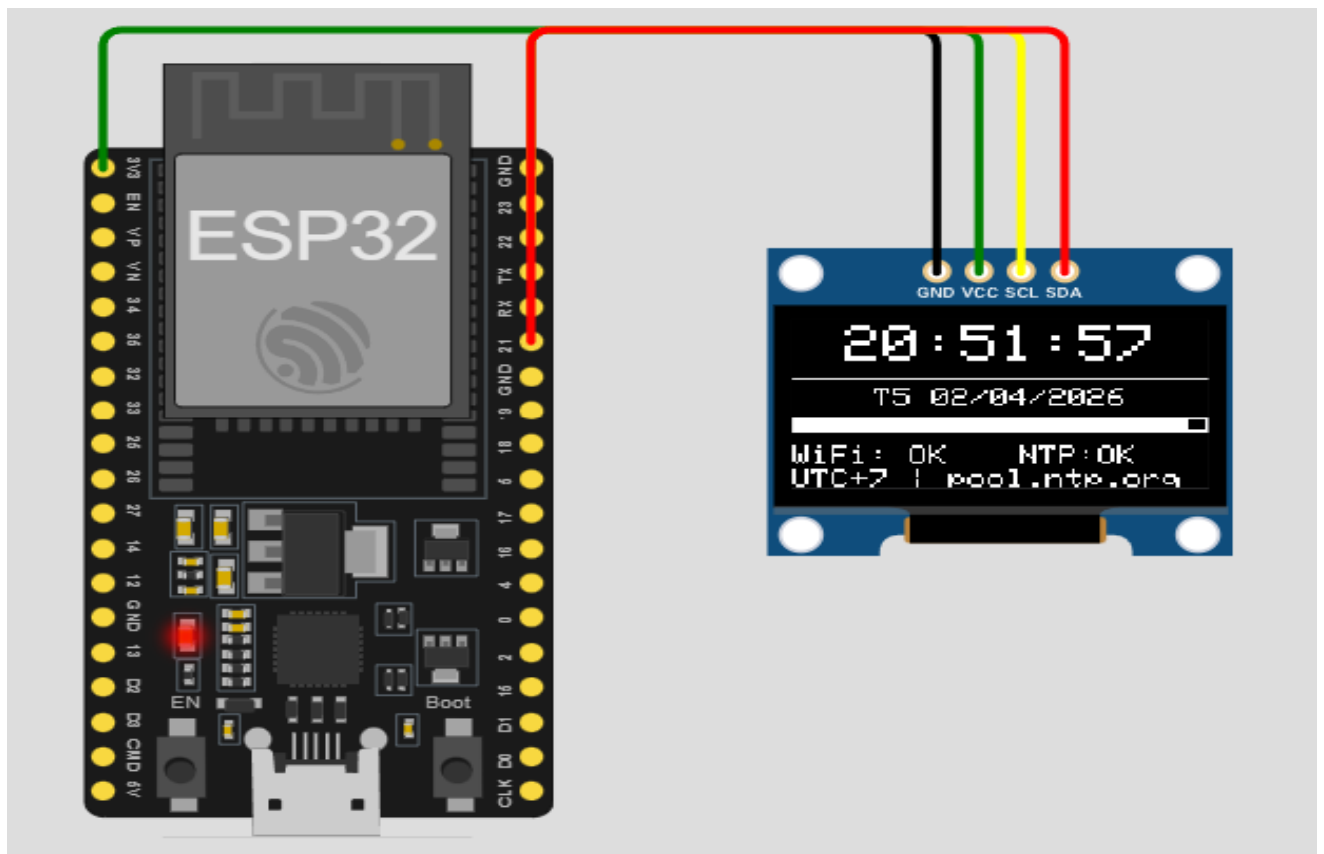
Cấu hình thời gian hiển thị trên thiết bị oled

```
display.setTextSize(1);  
char dateBuf[20];  
sprintf(dateBuf, "%s %02d/%02d/%04d",  
    DOW[t.tm_wday],  
    t.tm_mday, t.tm_mon+1, t.tm_year+1900);  
x = (SCREEN_W - strlen(dateBuf) * 6) / 2;  
display.setCursor(x, 26);  
display.print(dateBuf);
```

4.2. Kết quả hiển thị và độ chính xác của hệ thống

Demo:

Sau khi chèn thông tin mạng WiFi của bạn và sửa đổi các cấu hình thông tin thời gian như múi giờ,... bạn có thể nạp code vào ESP32 và kiểm tra



4.3 KẾT LUẬN

Qua quá trình triển khai đề tài, hệ thống đã đạt được các kết quả cụ thể sau:

1. **Độ chính xác cao:** Giải quyết triệt để vấn đề sai số tích lũy của các bộ dao động nội vi thông qua giao thức NTP, đảm bảo đồng bộ thời gian với độ trễ thấp trên nền tảng ESP32.
2. **Tối ưu phần cứng:** Loại bỏ nhu cầu sử dụng các module RTC vật lý (như DS3231), giúp giảm chi phí linh kiện và kích thước thiết bị mà vẫn duy trì được tính ổn định khi có kết nối Internet.
3. **Khả năng mở rộng:** Dữ liệu thời gian lấy từ NTP được lưu trữ hiệu quả dưới dạng biến cấu trúc (struct tm), tạo tiền đề để tích hợp vào các kịch bản điều khiển phức tạp như lập lịch tự động (Scheduling) hoặc ghi nhật ký sự kiện (Event Logging).

Hệ thống cho thấy khả năng hoạt động ổn định trong môi trường mạng Wi-Fi và có thể ứng dụng trực tiếp vào các dự án Smart Home hoặc trạm quan trắc IoT cần dữ liệu thời gian thực.

Tài liệu tham khảo

1. [ESP32 NTP: Đồng bộ ngày và giờ với mô hình Client Server - IoT Zone](#)
2. [ESP32 NTP Client-Server: Lấy Ngày và Giờ - Arduino IDE](#)
3. Chat AI Gemini
4. Link github demo đề tài:
[2025-2026.2.TIN4024.001/TEAM_04/Tran_Khanh/ESP32_Realtme_NTP at main · vvdung-husc/2025-2026.2.TIN4024.001](#)